Midterm Project Report

Evaluating Uncertainty Estimation Methods for Detecting Incorrect Predictions in Medical Image Classification

Ikechukwu Mgbemele · Senior Seminar I · Fisk University

Reporting period: Weeks 1–7 · Submitted 5 October 2026

1. Project Overview

Problem. A medical image classifier can perform well on the kind of images it was trained on and still fail on images that differ from them — images with more noise, blur, or weaker contrast. The dangerous part is not that the model is wrong. It is that the model is often wrong *and confident*, giving a clinician no signal that the prediction should be questioned.

Research question. How effectively can uncertainty estimation methods identify incorrect predictions when a medical image classifier is tested on degraded images that differ from its training data?

Objectives.

1. Train a medical image classifier and establish its accuracy on clean data.
2. Degrade the test images in controlled, gradable ways and measure how accuracy and confidence respond.
3. Implement three uncertainty estimation methods: softmax confidence, temperature scaling, and Monte Carlo Dropout.
4. Compare how well each method identifies predictions that are actually wrong, using AUROC and calibration error.
5. Determine which method gives the most reliable warning, and under which conditions.

Expected final product. A trained classifier, three implemented uncertainty methods, experiments on clean and degraded images, tables and figures of results, and a written analysis of which method is most effective at identifying unreliable predictions.

2. Work Completed: Weeks 1–7

Work was organised into five milestones. **Three are complete.** Each produced running code, saved numerical results, and figures. The sections below describe what was built, what was measured, and what each contributes.

Milestone	Completed	Status
1 — Environment, data pipeline, baseline classifier, clean evaluation	14–15 Sep	**Complete**
2 — Image degradations and behaviour under distribution shift	26 Sep	**Complete**
3 — Three uncertainty estimation methods	5 Oct	**Complete**
4 — Misclassification-detection metrics (AUROC, calibration error)	—	Not started
5 — Final comparison and written analysis	—	Not started

2.1 Tools and technologies

Component	Choice	Why
Language	Python 3.11	TensorFlow publishes no wheels for the machine's default Python 3.14, and 3.12 removed `distutils`, which TensorFlow 2.18 imports
Framework	TensorFlow 2.18 / Keras 3	As committed in the proposal
GPU	tensorflow-metal 1.2.0 on Apple M4	3.3× speedup; 1.8 vs 5.9 min/epoch
Analysis	NumPy, SciPy, scikit-learn, Matplotlib	Gaussian filtering, temperature optimisation, metrics, figures
Dataset	PathMNIST 64×64 (MedMNIST v2)	Public, no credentials, 9-class, standard benchmark
Version control	Git	3 commits; source, figures and metrics tracked

2.2 Milestone 1 — Baseline classifier (Weeks 1–4)

Dataset. PathMNIST: nine tissue classes from colorectal cancer histology slides — 89,996 training, 10,004 validation and 7,180 test images. I chose the **64×64** version over the default 28×28 because later milestones blur the test images, and at 28×28 there is too little spatial detail for blur to act as a gradual corruption. The archive is downloaded directly from Zenodo by `src/data.py`; I deliberately avoided the official `medmnist` package because it depends on PyTorch and torchvision, roughly 2.5 GB this TensorFlow project does not need.

One property of this dataset matters throughout the report: the **test split comes from a different hospital** than the training and validation splits. A mild distribution shift is therefore present before I add any artificial degradation.

Model. A VGG-style convolutional network with 1,244,009 parameters: four blocks of two 3×3 convolutions with batch normalisation and ReLU, each followed by max-pooling and dropout, with filter widths 32/64/128/256, then global average pooling, a 256-unit dense layer, and a linear output layer. Trained for 40 epochs (165 minutes), with Adam, early stopping and learning-rate reduction on validation loss. The best epoch by validation loss was epoch 34.

Two architectural decisions were made specifically to enable later milestones: the network outputs **raw logits rather than softmax probabilities**, because temperature scaling operates on logits; and **dropout is present from the start**, because Monte Carlo Dropout requires a model genuinely trained with dropout and cannot be retrofitted without retraining.

Results on clean data.

Metric	Validation	Test (different hospital)
Accuracy	0.9945	0.9348
Balanced accuracy	0.9945	0.9115
Macro F1	0.9944	0.9052
Macro AUROC (one-vs-rest)	1.0000	0.9950
Mean confidence	0.9964	0.9697

> **The result that motivates the whole project.** On validation, mean confidence (0.9964) and accuracy (0.9945) agree almost exactly — the model is well calibrated in-distribution. On the test split, accuracy falls to 0.9348 while confidence stays at 0.9697. The model is **overconfident by about 3.5 points** on data that is only mildly shifted. Of its 468 test errors, **96 were made at over 99% confidence** — cases a clinician would have no reason to double-check.

The errors are structured rather than random. Cancer-associated stroma is the weakest class at 0.582 recall, with 19% of it predicted as debris and 12% as smooth muscle; this is why debris shows perfect recall (1.000) but poor precision (0.644). Separately, 6% of normal colon mucosa is misread as adenocarcinoma epithelium.

2.3 Milestone 2 — Degradation study (Week 5)

I implemented three corruptions in `src/degradations.py`, applied to the **test set only** and never during training:

Corruption	Definition	What it models
Gaussian noise	add pixel-wise $N(0, \sigma)$	sensor noise, fast or low-dose acquisition
Gaussian blur	convolve with a Gaussian of width σ pixels	out-of-focus or lower-resolution scanner
Contrast reduction	$x' = (x - \text{mean}) \times \text{factor} + \text{mean}$	washed-out or poorly stained slide

Every corruption takes an explicit random seed. Without one, each uncertainty method in Milestone 3 would have been scored on a different corrupted test set and the comparison would have been meaningless.

Severity levels were calibrated, not guessed

Rather than choosing round numbers, I wrote `src/calibrate_severity.py` to sweep each corruption's parameter over a wide grid on a fixed 2,500-image subsample, measure accuracy at each value, and select the five levels whose accuracies fall most evenly between clean accuracy and a floor of 0.40. The floor sits well above chance (1/9 = 0.111) deliberately: a model that has collapsed to random guessing is not an interesting test of whether uncertainty estimates can flag its mistakes.

This mattered. The usable range proved far narrower than round numbers suggest — the model loses most of its accuracy between σ = 0.01 and 0.05 of noise. My initial guess of [0.02, 0.05, 0.09, 0.14, 0.20] would have placed severities 3, 4 and 5 all at or below chance, producing three identical and uninformative data points. The sweep also revealed that **accuracy is not monotonic** in corruption strength: under heavy contrast reduction accuracy falls to 0.0036, far *below* chance, which means the

model is not guessing randomly but confidently predicting one wrong class, and then rises again as the image collapses toward flat grey.

Corruption	s1	s2	s3	s4	s5
Gaussian noise (σ)	0.02	0.025	0.03	0.035	0.04
Gaussian blur (σ, px)	0.6	0.8	0.9	1.2	1.6
Contrast (factor)	0.85	0.8	0.7	0.65	0.6

Results

The full test set was evaluated at all 18 conditions (3 corruptions × 5 severities, plus clean). "Gap" is mean confidence minus accuracy; a perfectly calibrated model scores zero.

Corruption	Metric	s0	s1	s2	s3	s4	s5
Gaussian noise	accuracy	0.935	0.819	0.713	0.561	0.416	0.324
	mean confidence	0.970	0.933	0.903	0.878	0.870	0.878
	gap	+0.035	+0.115	+0.191	+0.317	+0.454	**+0.554**
Gaussian blur	accuracy	0.935	0.818	0.691	0.618	0.499	0.374
	mean confidence	0.970	0.930	0.893	0.881	0.854	0.857
	gap	+0.035	+0.112	+0.202	+0.264	+0.355	**+0.483**
Contrast	accuracy	0.935	0.770	0.705	0.578	0.437	0.328
	mean confidence	0.970	0.928	0.912	0.856	0.814	0.819
	gap	+0.035	+0.158	+0.207	+0.278	+0.376	**+0.490**

Accuracy collapses; confidence barely moves. Under noise, accuracy falls 61 points while mean confidence falls 9. At severity 5 the model reports 88% confidence while being right 32% of the time.

Confidence can rise while accuracy falls. From noise severity 4 to 5, accuracy drops from 0.416 to 0.324 while mean confidence *increases* from 0.870 to 0.878. The same reversal appears under blur and contrast. Confidence is not simply a lagging indicator of accuracy; the two can move in opposite directions.

Confidence becomes less discriminative. On clean data mean confidence is 0.983 when correct and 0.777 when wrong. Under corruption the two converge, because confidence on wrong predictions actually *rises* while confidence on correct ones falls:

Separation (confidence correct − incorrect)	s0	s1	s2	s3	s4	s5
Gaussian noise	0.207	0.150	0.131	0.099	0.093	0.090
Gaussian blur	0.207	0.140	0.122	0.106	0.084	0.086
Contrast	0.207	0.117	0.107	0.102	0.066	**0.026**

The corruption type matters. Errors made above 99% confidence rise from 96 on clean data to 1,644 under noise severity 5 — 34% of all errors at that severity — and blur behaves the same way. Contrast does the opposite: its share of above-99%-confidence errors *falls* to 8.4%. Washing the image out makes the model diffusely uncertain, whereas noise and blur make it confidently wrong. The coarse claim "the

model becomes overconfident under shift" is therefore too imprecise; the mechanism differs by corruption.

Pixel-level change does not predict damage. Measuring how far each severity-5 corruption moves the image shows noise is roughly 1.9× more damaging per unit of pixel change than blur, suggesting the network depends on fine texture that noise destroys and blur merely softens.

Corruption (severity 5)	RMSE vs clean	Accuracy	Accuracy lost per unit RMSE
Gaussian noise	0.040	0.324	**15.4**
Contrast	0.064	0.328	9.5
Gaussian blur	0.068	0.374	8.2

2.4 Milestone 3 — Uncertainty estimation methods (Weeks 6–7)

I implemented the three methods in `src/uncertainty.py` and ran all of them over all 18 conditions.

Method	What it does	Can it change the prediction?
Softmax confidence	unmodified network output	— (it is the prediction)
Temperature scaling	divide logits by a scalar T	**No** — monotonic in the logits
Monte Carlo Dropout	average 30 stochastic forward passes	**Yes**

Temperature scaling. $T = \mathbf{1.2759}$, fitted by minimising negative log-likelihood on one half of the validation split and reported on the other half, so the benefit is never measured on the data that produced it. The test set was never used to fit it. On clean validation the model was already nearly calibrated (gap +0.0024), so T had almost nothing to correct in-distribution — which is precisely the honest deployment setting, since clean data is the only data available for calibration.

Monte Carlo Dropout. 30 stochastic forward passes per image, averaged. In addition to the mean prediction I compute *mutual information*, the gap between the entropy of the averaged prediction and the mean entropy of the individual predictions, which isolates disagreement between the sampled sub-networks rather than overall ambiguity.

Results: overconfidence gap by method

Corruption	Method	s0	s1	s2	s3	s4	s5
Noise	softmax	+0.035	+0.115	+0.191	+0.317	+0.454	+0.554
	temperature	+0.025	+0.095	+0.163	+0.285	+0.421	+0.521
	MC Dropout	+0.019	**+0.048**	**+0.115**	**+0.212**	**+0.331**	**+0.439**
Blur	softmax	+0.035	+0.112	+0.202	+0.264	+0.355	+0.483
	temperature	+0.025	+0.090	**+0.170**	**+0.229**	+0.318	+0.441
	MC Dropout	+0.020	**+0.085**	+0.183	+0.231	**+0.286**	**+0.403**
Contrast	softmax	+0.035	+0.158	+0.207	+0.278	+0.376	+0.490
	temperature	+0.025	+0.136	**+0.180**	**+0.237**	**+0.329**	+0.435
	MC Dropout	+0.019	**+0.131**	+0.200	+0.270	+0.330	**+0.418**

Every method improves calibration, and none of them comes close to fixing it. The best method at noise severity 5 still reports 77% confidence while being right 37% of the time. Recalibration reduces overconfidence; it does not make the model know when it is wrong.

No method wins everywhere. MC Dropout dominates under noise at every severity. Under blur and contrast the ranking flips with severity: temperature scaling is better at severities 2–3, MC Dropout at 4–5. Any claim that one method is simply "best" would be an artefact of which corruption was tested.

MC Dropout changes accuracy; temperature scaling cannot. I verified that temperature scaling changed **0** of 7,180 predictions across all 18 conditions, exactly as the mathematics requires. MC Dropout raises accuracy under noise (0.324 to 0.370 at severity 5) but *lowers* it under blur severity 2 (0.691 to 0.679) and contrast severity 3 (0.578 to 0.547). Averaging over sub-networks is not free.

Mutual information is the one quantity that behaves consistently. It rises monotonically with severity under all three corruptions (0.031 on clean data to 0.122 for noise, 0.124 for blur and 0.136 for contrast) without a single reversal. Unlike confidence, which can rise while accuracy falls, disagreement between dropout sub-networks tracks corruption consistently, making it a candidate shift detector even where it fails as an error detector.

> **A measurement trap I identified and avoided.** Temperature scaling appears to improve the separation between confidence-when-right and confidence-when-wrong, from 0.207 to 0.240 on clean data. This is an artefact. Because temperature scaling is monotonic it barely reorders predictions: measured Spearman correlation with the unscaled confidences is **0.9994**, and error-detection AUROC moves by less than **0.005**. It fixes *calibration* (how large the numbers are) but cannot improve *discrimination* (which predictions get flagged first). Only MC Dropout can genuinely reorder. This yields a falsifiable prediction for Milestone 4, recorded here before the measurement is taken: AUROC for temperature scaling will be essentially identical to plain softmax.

An oracle upper bound

I also fitted a separate "oracle" temperature on each condition. It is not deployable, because it requires labelled data from the corruption being faced, but it bounds what temperature scaling could ever achieve.

	Clean	Noise s3	Noise s5
Oracle T^*	1.89	3.64	5.67
Oracle gap	−0.004	+0.041	**+0.109**

Two things follow. The oracle temperature for the *clean* test set is 1.89 against the 1.28 fitted on validation, so the cross-hospital shift alone already makes the validation-fitted temperature too small. And even the oracle cannot reach a zero gap at high severity: a single scalar is not sufficient to recalibrate a badly shifted model, even when fitted directly on the shift.

3. Evidence of Progress

Repository. The project is a Git repository with 3 commits containing 1,808 lines of Python across 11 modules, 8 figures and 6 metrics files. The 1 GB dataset, the 14 MB trained model and the raw prediction files are excluded, since all are regenerable from the source.

Module	Purpose
`src/config.py`	all settings, paths, severity levels
`src/data.py`	download, loading, input pipelines, augmentation
`src/model.py`	baseline CNN (1,244,009 parameters)
`src/train.py`	training loop, callbacks, curves
`src/evaluate.py`	clean-split metrics, logits, confusion matrix
`src/degradations.py`	the three corruptions and severity registry
`src/calibrate_severity.py`	empirical selection of severity levels
`src/evaluate_degraded.py`	metrics, logits and figures under degradation
`src/uncertainty.py`	the three uncertainty methods
`src/fit_temperature.py`	fits T on held-out validation data
`src/evaluate_uncertainty.py`	all methods over all conditions

Saved numerical results. 74 prediction files (18 degraded logit sets and 54 predictive distributions, one per method per condition) and 6 JSON metrics files covering training history, clean evaluation, severity calibration, degraded evaluation, the fitted temperature and the uncertainty comparison. Saving full probability matrices rather than summaries means any uncertainty score can be computed in Milestone 4 without re-running the network, and every method is scored on byte-identical inputs.

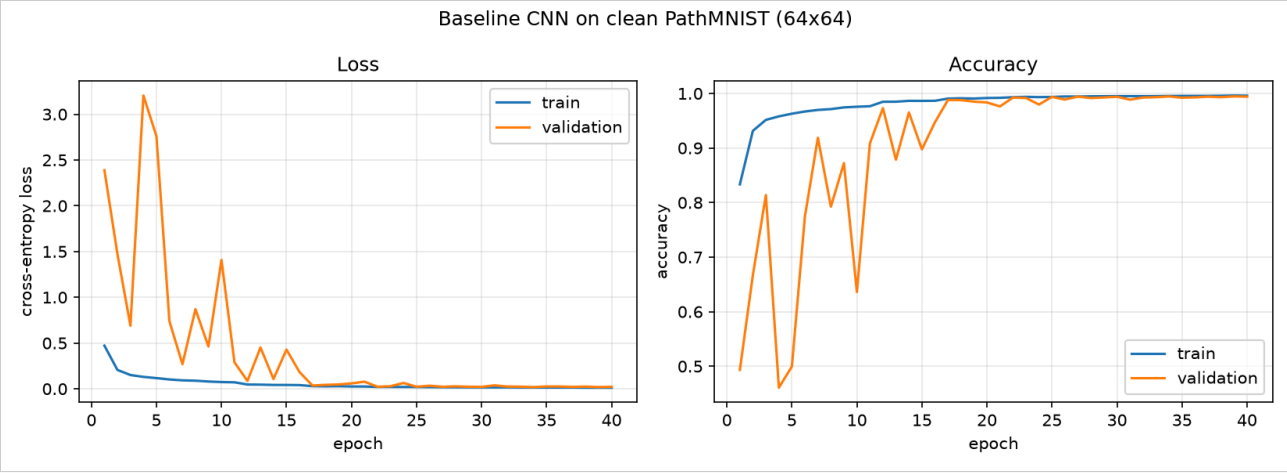


Figure 1. Baseline training over 40 epochs. The large validation oscillation through roughly epoch 16 is learning-rate driven and settles once the scheduler reduces the rate; the earlier chance-level region is the BatchNorm problem described in Section 5.

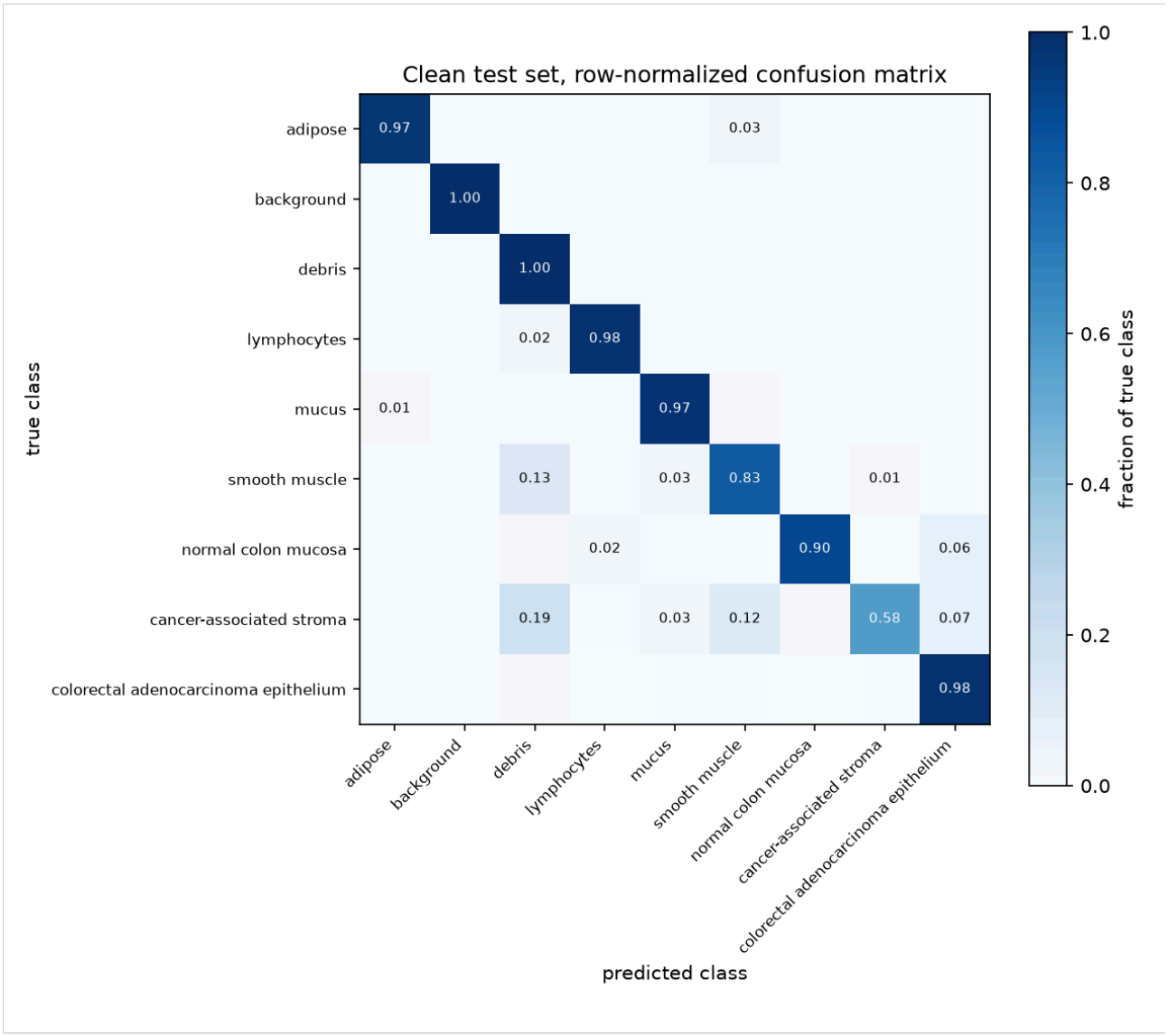


Figure 2. Row-normalised confusion matrix on the clean test set. Errors are structured: cancer-associated stroma is correct only 58% of the time, with 19% going to debris and 12% to smooth muscle.

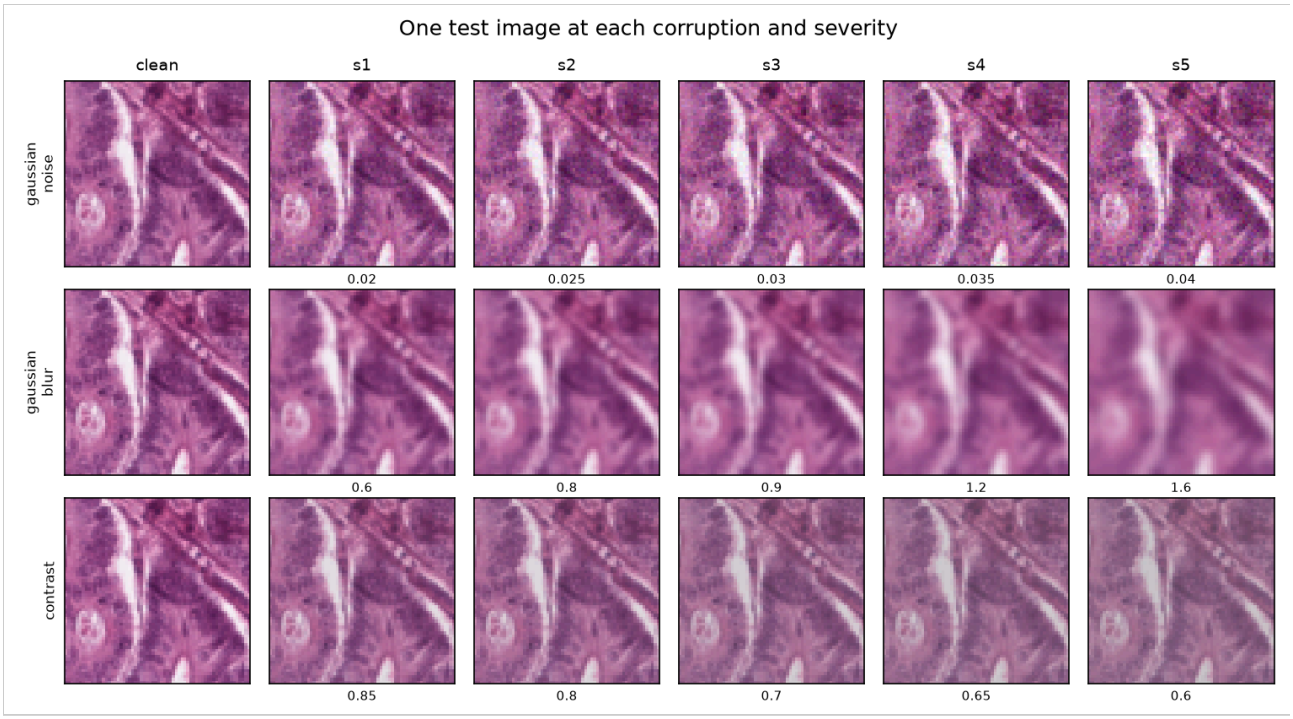


Figure 3. One test image at each corruption and severity. Even at severity 5 the tissue structure remains readable to a human eye, yet classifier accuracy falls to roughly 0.32. These are not images a person would reject as unusable, which is what makes the failure silent.

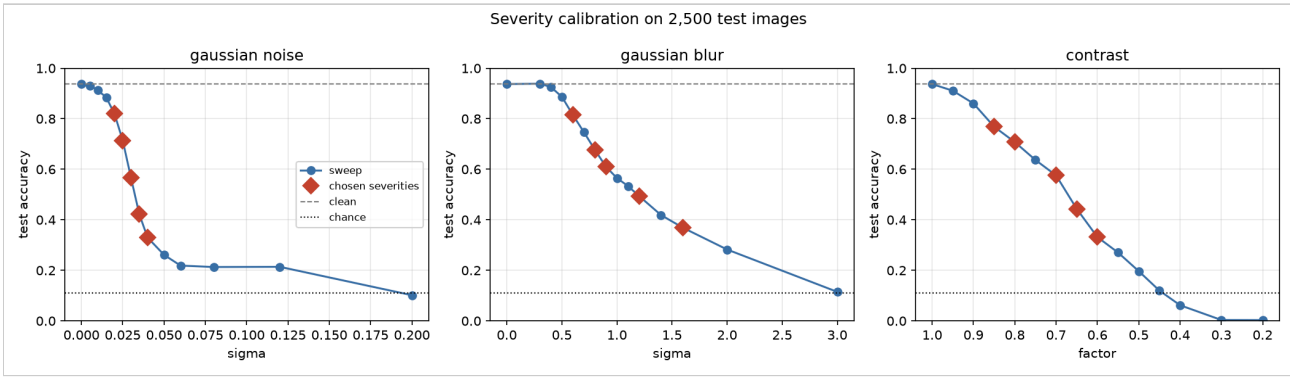


Figure 4. Severity calibration on 2,500 test images. Red diamonds mark the chosen levels; the dotted line is chance. The noise panel shows why round-number severities would have failed: all five usable levels sit inside a narrow cliff between $\sigma = 0.02$ and 0.04, beyond which a plateau means further corruption changes nothing.

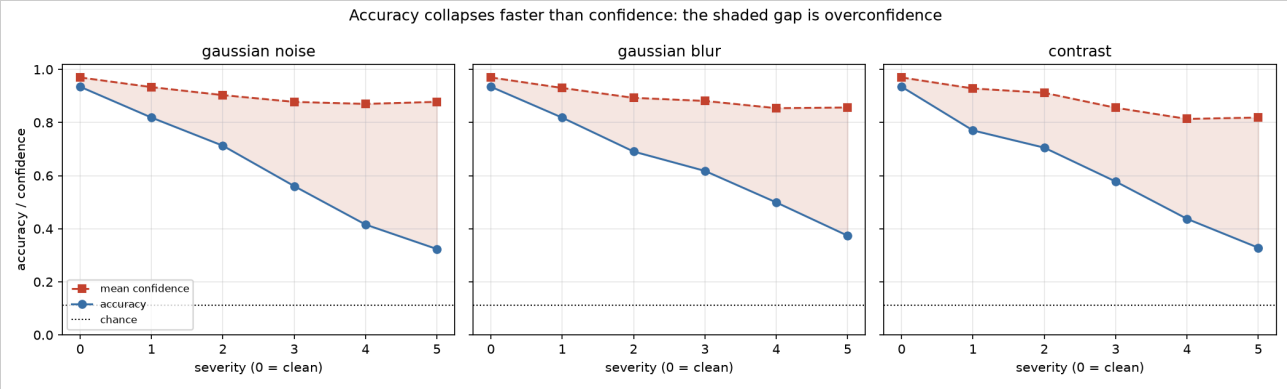


Figure 5. Accuracy and mean confidence against severity. The shaded region is the overconfidence gap, widening from +0.035 on clean images to +0.554 under noise at severity 5.

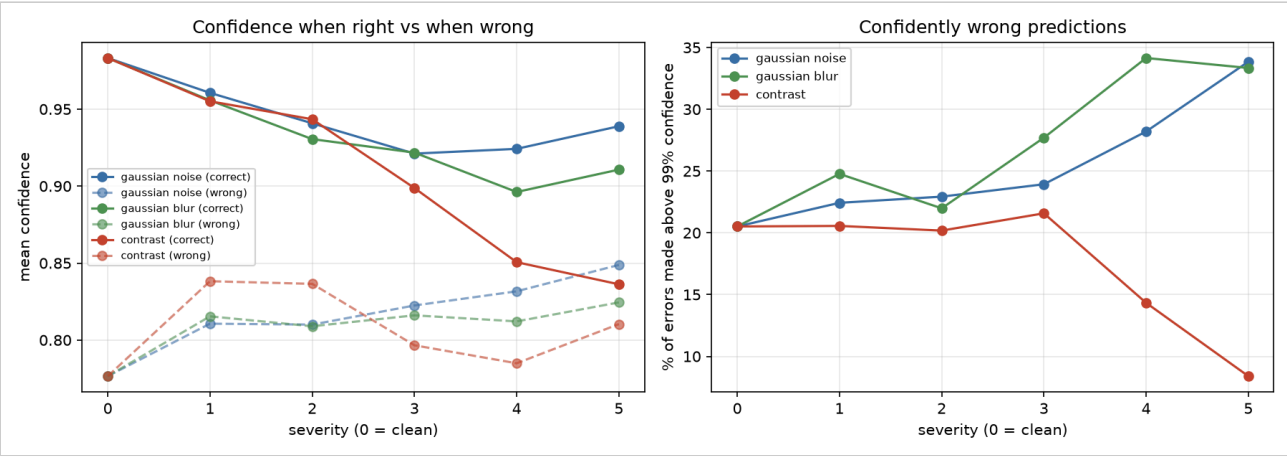


Figure 6. Left: mean confidence on correct (solid) versus incorrect (dashed) predictions, converging as severity rises. Right: the share of errors made above 99% confidence, which rises for noise and blur but falls for contrast — evidence that the failure mode depends on the corruption.

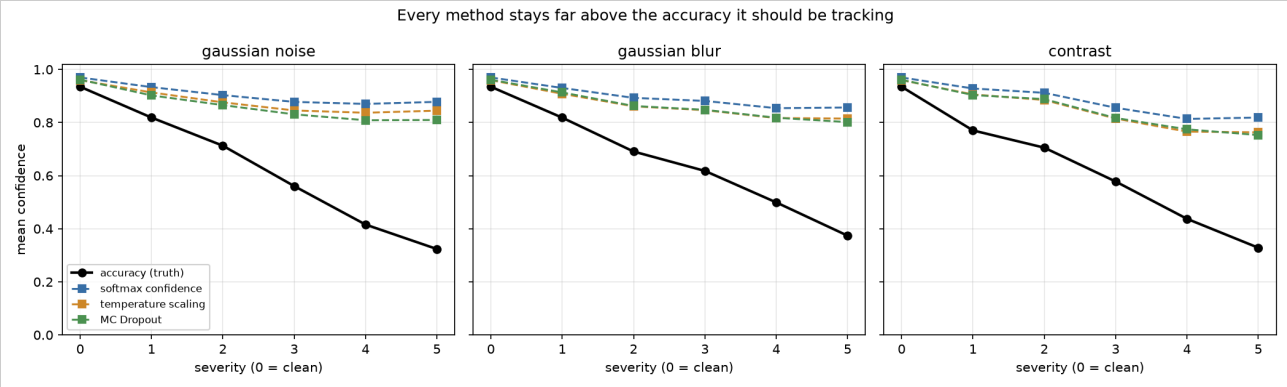


Figure 7. Mean confidence of each uncertainty method against true accuracy (black). All three methods cluster far above the accuracy they should be tracking.

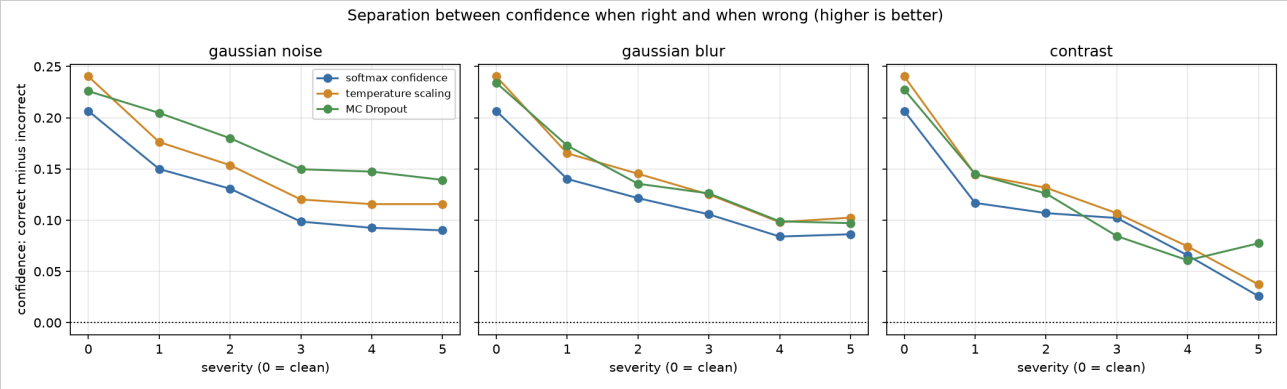


Figure 8. Separation between confidence when right and when wrong, by method. MC Dropout preserves the most separation under noise; under contrast all three collapse toward zero.

4. Progress Compared with the Original Proposal

My proposal committed to specific tasks. Their status:

Commitment in the proposal	Status
Use Python with TensorFlow or Keras	**Complete — TensorFlow 2.18 / Keras 3**
Use a publicly available medical image classification dataset	**Complete — PathMNIST 64×64**
Train or fine-tune a standard image classification model	**Complete — CNN trained from scratch, 0.9348 test accuracy**
Evaluate the model on the normal test set	**Complete**
Create altered test images with Gaussian noise, blur and contrast changes	**Complete — 5 calibrated severities each**
Keep altered images out of training	**Complete — test split only**
Compare softmax confidence, temperature scaling and Monte Carlo Dropout	**Complete — all three implemented and run on all conditions**
Measure accuracy, with the main focus on identifying wrong predictions	**In progress — accuracy done; formal error detection is Milestone 4**
Use AUROC and calibration error to compare the methods	**In progress — Milestone 4**
Produce tables, graphs and an analysis of which method is most effective	**In progress — 8 figures and all tables produced; final analysis is Milestone 5**

Am I on schedule? Yes. Every experimental component the proposal described is built and running. What remains is the comparison layer: turning the saved predictions into AUROC and calibration-error numbers, and writing the analysis. Because all 54 predictive distributions are already saved to disk, Milestone 4 requires no retraining and no re-running of the network.

Changes to the original plan, and why

- **Trained from scratch rather than fine-tuning a pretrained model.** The proposal allowed either. Pretrained ImageNet models expect 224×224 inputs, which would have meant upscaling 64×64

histology tiles by 3.5× and a far more expensive model for no clear benefit. A compact CNN reaches 0.9348 test accuracy and trains in under three hours.

- **Added an empirical severity-calibration step.** Not in the original plan. It became necessary once I found that hand-picked severities placed most levels below chance. This is now a methodological contribution rather than an implementation detail.
- **Added an oracle temperature as an upper bound.** Not in the original plan. Without it, a weak result for temperature scaling would be ambiguous between "the method is weak" and "the fitted T was poor".
- **Deliberately excluded contrast and brightness from training augmentation.** Not anticipated in the proposal. Training with contrast jitter would have made the model robust to precisely the corruption the project studies, confounding the entire experiment.

5. Challenges and Solutions

5.1 TensorFlow would not install on the available Python

Problem. The machine's default Python is 3.14, for which TensorFlow publishes no wheels. Python 3.12 also failed, because TensorFlow 2.18 imports `distutils`, which 3.12 removed from the standard library, and the compatibility shim did not reliably activate at interpreter startup.
Impact. Blocked all work until resolved.
Solution. Rebuilt the environment on Python 3.11, which still ships `distutils`. **Resolved.**

5.2 GPU plugin incompatible with the installed TensorFlow

Problem. `tensorflow-metal` 1.2.0 is ABI-incompatible with TensorFlow 2.21 and broke `import tensorflow` outright with a dynamic-linking error.
Impact. Without GPU acceleration, training would have taken about three times longer (5.9 versus 1.8 minutes per epoch).
Solution. Pinned TensorFlow 2.18 with tensorflow-metal 1.2.0. Because this project is about confidence calibration, I verified the GPU numerically before trusting it: with identical weights, GPU and CPU forward passes agreed to a maximum absolute difference of 1.8×10^{-7} with 100% agreement on the predicted class, and gradient norms matched to six significant digits. **Resolved.**

5.3 Validation accuracy stuck at chance while training looked healthy

Problem. Early in training, accuracy on the training set climbed normally while validation accuracy sat at chance (1/9), with the model predicting a single class for all 10,004 validation images.
Impact. This looked exactly like a data bug — mismatched images and labels, or a corrupted split — and would have invalidated every downstream result.
Diagnosis. I evaluated the same model using batch statistics instead of BatchNorm's stored moving averages and obtained 0.65 on training data and 0.64 on validation: nearly identical and both healthy. Only inference mode collapsed, and it collapsed on the *training* data too. A corrupted validation split cannot explain a failure that also appears on training data, which exonerated the data.
Cause. Keras defaults BatchNorm momentum to 0.99. The true activation variance in this network is far below BatchNorm's initial value of 1.0, so the leftover initialisation still dominated the stored moving variance after hundreds of updates, compounding across nine BatchNorm layers. This was not merely cosmetic: checkpointing and early stopping both select on validation loss, so unreliable early validation numbers corrupt model selection.
Solution. Set momentum to 0.9. Validation accuracy after two identical epochs rose from 0.109 to 0.640, and after full training the stored moving variance matched the true activation variance to within 0.6%. **Resolved.**

5.4 Hand-picked corruption severities were unusable

Problem. My first severity levels placed three of five noise levels at or below chance. A second attempt, selecting levels by closest accuracy to a target, chose the identity transform (no corruption at all) as contrast severity 1, because accuracy is not monotonic in corruption strength.
Impact. Would have produced several identical, uninformative data points and an invalid severity scale.

Solution. Wrote a calibration script that sweeps each parameter, then selects the first value to cross each accuracy target while walking forward through the grid, which guarantees levels strictly increasing in strength and never reuses the clean image. **Resolved.**

5.5 Monte Carlo Dropout would have silently corrupted BatchNorm

Problem. The usual way to keep dropout active at prediction time is to call the model with `training=True`. That flag is global, so it would also have switched BatchNormalization into batch-statistics mode — the same failure as Challenge 5.3.
Impact. Every MC Dropout uncertainty estimate would have been wrong, with no error message. I measured the counterfactual: `training=True` changes 5.9% of predictions outright.
Solution. Subclassed Keras's Dropout layer to force itself active regardless of the flag, and cloned the model replacing only the dropout layers, leaving BatchNorm untouched. I verified the surgery by building a clone with the dropout rate forced to zero: it reproduces the original model's output *exactly* (difference 0.0), proving BatchNorm still uses its moving statistics. **Resolved.**

5.6 Monte Carlo Dropout is not reproducible on this hardware

Problem. Seeding does not make MC Dropout deterministic on the Metal GPU backend. Two runs with an identical seed produced accuracies of 0.9410 and 0.9435, with individual predicted probabilities differing by as much as 0.26.
Impact. Small reported differences between MC Dropout and the other methods could be noise rather than signal.
Solution. Quantified the noise floor instead of ignoring it: across five seeds accuracy spanned 0.9415 to 0.9450, a spread of 0.0035. MC Dropout's advantage under noise (+3.5 to +6.1 accuracy points) is an order of magnitude above this, so the main conclusions hold, but differences of about one point are now reported as ties. **Resolved as a documented limitation** rather than eliminated; forcing determinism on this backend is not straightforward.

5.7 A metric that would have produced a false conclusion

Problem. Temperature scaling appeared to improve the separation between confidence-when-right and confidence-when-wrong, which would have supported the conclusion that it improves error detection.
Impact. This would have been a wrong finding in the final report.
Solution. I checked whether temperature scaling actually reorders predictions. Spearman correlation with the unscaled confidences is 0.9994 and error-detection AUROC moves by less than 0.005, so the apparent gain is an artefact of nonlinear squashing rather than better detection. The limitation is now documented in the code so the metric is not misread. **Resolved.**

6. Current Project Status

Three of five milestones are complete. The project has a working classifier, a validated degradation pipeline, and all three uncertainty methods implemented and run across every experimental condition. The experimental infrastructure is finished; what remains is measurement and analysis.

Major outcomes so far:

- A trained classifier reaching 0.9348 accuracy on the clean test set and 0.9945 on validation.

- Quantified evidence that the model is overconfident even on clean test data, with 96 of its 468 errors made at over 99% confidence.
- A calibrated degradation benchmark of 18 conditions showing the overconfidence gap growing from $+0.035$ to $+0.554$, with confidence sometimes *rising* as accuracy falls.
- Evidence that the failure mode depends on corruption type: noise and blur produce confidently wrong predictions, contrast produces diffuse uncertainty.
- All three uncertainty methods implemented, with the finding that each improves calibration, none fixes it, and no method wins under every corruption.
- Mutual information identified as the only measured quantity that rises monotonically with corruption severity.
- 54 saved predictive distributions, so the remaining analysis requires no recomputation.

What is incomplete: the formal misclassification-detection metrics (AUROC and expected calibration error) that answer the research question directly, and the final written analysis. The descriptive evidence gathered so far strongly suggests what those metrics will show, but it does not substitute for measuring them.

7. Plan for the Second Half of the Semester

Milestone 4 — Misclassification-detection metrics

This is the milestone that answers the research question. For each method at each of the 18 conditions I will compute:

- **AUROC for misclassification detection**, the headline metric: how well does each uncertainty score rank wrong predictions above right ones? This will test the prediction recorded in Section 2.4 — that temperature scaling matches plain softmax almost exactly, because it cannot reorder predictions.
- **Expected calibration error**, with reliability diagrams. AUROC measures ranking within a condition and will not by itself reveal that confidence is globally too high; the two metrics answer different questions and both are needed.
- **Risk–coverage analysis**: if the model abstains on its least confident x%, how much does accuracy recover? This is the form a clinical workflow would actually use, and it converts the abstract comparison into a decision a hospital could act on.
- **Mutual information as an error-detection score**, compared against maximum probability, since it is the only quantity that tracked severity monotonically.

Milestone 5 — Final comparison and written analysis

- Consolidate all results into a single comparison across methods, corruptions and severities.
- Produce the final figure set and results tables.
- Write the analysis: which method gives the most reliable warning, under which conditions, and at what cost. MC Dropout is 30× more expensive at inference than the alternatives, which is a real consideration for deployment.
- State the limitations honestly: synthetic corruptions are not real scanner artefacts, the study uses one architecture and one dataset, only contrast reduction was tested rather than increase, and the severity

levels are calibrated to this particular model.

Risks for the remainder

The main technical risk is low, because no further training is required and the saved predictions make the remaining analysis cheap to run and re-run. The more realistic risk is interpretive: with three methods, three corruptions, six severity levels and several metrics, the results form a large grid, and the conclusion must be a clear answer to the research question rather than an undigested table. Evidence already gathered — that no method wins everywhere, and that the corruption type changes the failure mode — suggests the honest answer will be conditional rather than a single winner, and the final analysis needs to present that clearly instead of overstating a result.